Deep Research

Presenters: Liangtao Dai, Tian Yu, Ziyu Xuan

Deep Research

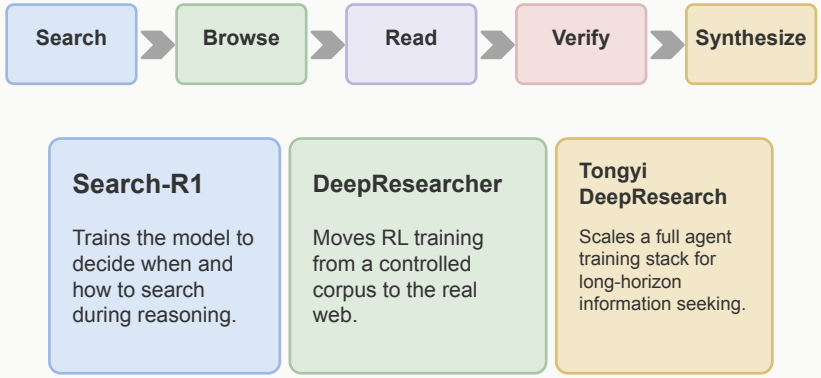
From retrieval-augmented QA to long-horizon research agents

Shared motivation

How can an LLM move from retrieve-once QA to a research agent that can search, read, verify, and synthesize information over multiple steps?

- Standard retrieval usually helps with one-shot question answering, not long-horizon research workflows.
- Deep research requires iterative search, webpage reading, evidence checking, and synthesis across multiple sources.
- All three papers ask how these behaviors can become part of the learned policy itself.

What deep research needs



Together, the three papers describe a progression from tool-assisted retrieval to trained deep research agents.

Deep Research

From retrieval-augmented QA to long-horizon research agents

Problem and background

Why plain retrieval is not enough for deep research

Standard RAG usually assumes:

- A fixed corpus is already available and relevant information can be retrieved directly.
- One query or one retrieval step is often enough before generation begins.
- The model does not need to explore noisy webpages or compare competing sources.
- Search remains an external helper, rather than a behavior learned inside reasoning.

Deep research usually requires:

- Multi-step search and browsing, because useful evidence may be scattered across many pages.
- Reading, filtering, and summarizing long web content instead of only ranking short passages.
- Cross-checking evidence when sources are incomplete, outdated, or inconsistent.
- A learned policy that knows when to search again, when to stop, and how to synthesize.

How the three papers line up

Search-R1 → *Learn Search behavior Inside reasoning via RL*

DeepResearcher → *show that training on the real Web matters*

Tongyi DeepResearch — *scale the full agent Training recipe*

Search-R1

Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning

Bowen Jin et al. | COLM 2025
Updated class presentation with PPO vs. GRPO discussion

Main message

SEARCH-R1 does not only give an LLM a search tool. It uses reinforcement learning to train the model to decide when to search, what to search for, and how to use retrieved evidence during reasoning.

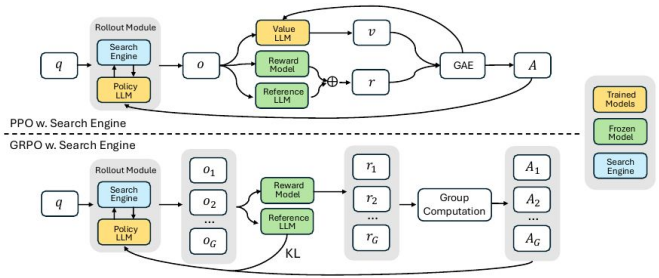


Figure 1: Demonstration of PPO and GRPO training with the search engine (SEARCH-R1). During the rollout, LLMs can conduct multi-turn interactions with the search engine.

Main framework figure from the paper

Roadmap

- First, I will explain the problem: why search prompting or one-shot retrieval is still incomplete for reasoning-intensive QA.
- Then I will summarize the paper's main contributions and why they matter beyond this single benchmark setting.
- Next, I will show the SEARCH-R1 framework and the most important technical choices, including multi-turn search and retrieved-token masking.
- After that, I will cover the experiment setup, the main benchmark result, and one dedicated slide on PPO versus GRPO.
- Finally, I will discuss additional analysis, limitations, future directions, and the overall takeaway.

Question to keep in mind

Does RL-trained search behavior outperform simply adding retrieval or prompting the model to use search at inference time?

Key paper evidence

The talk will focus on Figure 1 for the method, Table 2 for the main result, Figure 2 plus Table 3 for PPO/GRPO analysis, and Table 4 for masking.

Presentation style

I will not cover every appendix detail. I will focus on the main ideas, the most important technical design choices, and the evidence that supports the paper's claims.

Problem and motivation

Why search access alone is not enough

- LLMs often need external and up-to-date information for open-domain QA and multi-hop reasoning tasks.
- A common fix is to add retrieval, either as one-shot RAG or as prompted search during inference.
- But this still leaves an important gap: the model is not explicitly trained to decide when to search, what query to issue, and how to use the returned evidence in later reasoning steps.
- The paper argues that search behavior itself should be optimized as part of the reasoning policy, not treated as a fixed helper module.

Why this matters

This is a shift from “connect retrieval to an LLM” to “learn retrieval behavior inside reasoning.” That is the conceptual difference between SEARCH-R1 and standard RAG-style pipelines.

Core research question

Can an LLM learn search-and-reason trajectories directly from simple outcome reward?

Relation to RAG

RAG usually retrieves first and then generates. SEARCH-R1 interleaves generation and search multiple times in one trajectory.

Existing works

Why search access alone is not enough

Integrating LLMs with search engines:

1. Retrieval-augmented generation (RAG)
2. Treating the search engine as a tool

Reinforcement Learning (RL) for LLMs

1. Improve logical inference and problem-solving skills by learning from experience and feedback
2. The models learn complex reasoning capabilities even when trained solely on the outcome rewards

Limitation

1. LLM is not optimized to learn how to interact effectively with search engines during training
2. Prompting-based approaches often struggle to generalize

Challenges

1. RL Framework and Stability
2. Multi-Turn Interleaved Reasoning and Search
3. Reward Design

Major contributions and why they matter

Contribution 1
SEARCH-R1 formulates search as a learned action inside reasoning rollouts, instead of a fixed retrieval step or a prompt-only behavior.

Contribution 2
The method uses a simple RL design: multi-turn search rollout, final-answer reward, and support for both PPO and GRPO optimization.

Contribution 3
It introduces retrieved-token loss masking so optimization focuses on model-generated tokens instead of accidentally optimizing over copied retrieved text.

Why important scientifically?
The paper suggests that better search-augmented reasoning may require policy learning, not only better retrievers or stronger prompting templates.

Why important technically?
The framework is simple enough to reproduce: no complex reward model, no hand-designed intermediate supervision, and no unusual architecture changes.

Challenges in RL to search-and-reasoning scenarios
1. RL Framework and Stability
2. Multi-Turn Interleaved Reasoning and Search
3. Reward Design

Method overview

SEARCH-R1 trains reasoning and search jointly

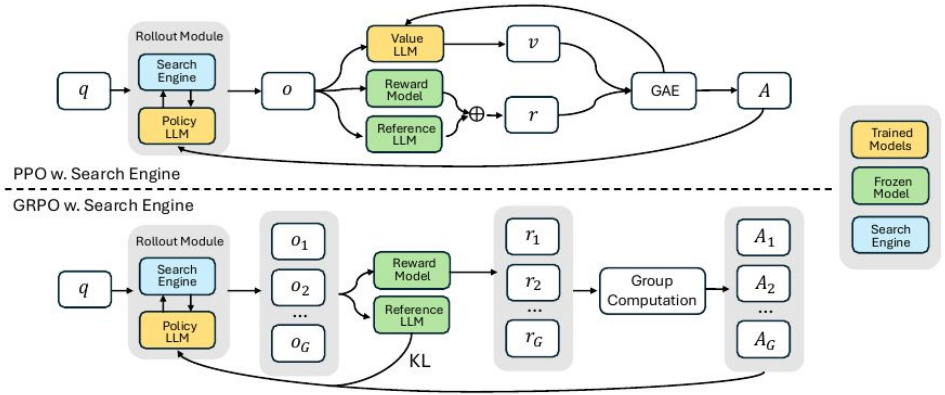


Figure 1: Demonstration of PPO and GRPO training with the search engine (SEARCH-R1). During the rollout, LLMs can conduct multi-turn interactions with the search engine.

Figure 1 from the paper

- The model begins by generating reasoning tokens for the question.
- Whenever it decides more information is needed, it emits a search query and calls the search engine.
- The retrieved passages are appended to the same rollout, and the model continues reasoning with that evidence.
- Reinforcement learning then optimizes the full search-augmented trajectory based on final answer correctness.

Main conceptual point

Search is part of the policy loop. It is not only a preprocessing step before generation starts.

Most important technical details

Algorithm 1 LLM Response Rollout with Multi-Turn Search Engine Calls

Require: Input query x, policy model π_θ, search engine $\mathcal{R}$, maximum action budget B.
Ensure: Final response y.

```
Initialize rollout sequence y ← ∅
Initialize action count b ← 0
while b < B do
    Initialize current action LLM rollout sequence y_b ← ∅
    while True do
        Generate response token y_t ~ π_θ(· | x, y + y_b)
        Append y_t to rollout sequence y_b ← y_b + y_t
        if y_t in [</search>, </answer>, <eos>] then break
        end if
    end while
    y ← y + y_b
    if <search> </search> detected in y_b then
        Extract search query q ← Parse(y_b, <search>, </search>)
        Retrieve search results d = R(q)
        Insert d into rollout y ← y + <information>d</information>
    else if <answer> </answer> detected in y_b then
        return final generated response y
    else
        Ask for rethink y ← y+ "My action is not correct. Let me rethink."
    end if
    Increment action count b ← b + 1
end while
return final generated response y
```

Algorithm 1 from the paper

Answer the given question. You must conduct reasoning inside <think> and </think> first every time you get new information. After reasoning, if you find you lack some knowledge, you can call a search engine by <search> query </search>, and it will return the top searched results between <information> and </information>. You can search as many times as you want. If you find no further external knowledge needed, you can directly provide the answer inside <answer> and </answer> without detailed illustrations. For example, <answer> xxx </answer>. Question: question.

Table 1: Template for SEARCH-R1. question will be replaced with the specific question during training and inference.

1) Multi-turn search rollout

The model can search more than once before giving the final answer. This is especially useful for multi-hop questions that require chained evidence.

2) Outcome-based reward

The main reward depends on whether the final answer is correct. The framework does not rely on dense intermediate annotations.

3) Retrieved-token masking

Retrieved passages are inserted into the rollout, but those retrieved tokens are masked out in the policy objective. The training signal focuses on what the LLM itself generates.

Why this matters

The method is conceptually simple, but these design choices make search-augmented RL training stable enough to work in practice.

Experimental setup

Models
Qwen2.5-3B and Qwen2.5-7B, each evaluated in base and instruct versions.

Datasets
Seven QA benchmarks spanning both general QA and multi-hop QA.

Retriever
The paper uses 2018 Wikipedia plus an E5 retriever, with three retrieved passages for retrieval-based methods.

Baselines
Direct inference, CoT, RAG, IRCoT, Search-o1, SFT, R1, and rejection sampling.

Comparison principle
The retriever, corpus, and pretrained models are aligned whenever possible so the comparison is about search behavior, not mismatched infrastructure.

- This setup separates three ideas clearly: no retrieval, inference-time retrieval, and learned search behavior.
- That makes the comparison more informative than simply comparing one method against one baseline.
- So the setup is broad enough to test the paper’s central claim in a fairly controlled way.

Main empirical findings

Table 2 provides the main evidence for the paper's central claim

Table 2: Main results. The best performance is set in bold. †/⋆ represents in-domain/out-domain datasets.

Methods	General QA			Multi-Hop QA				
	NQ†	TriviaQA⋆	PopQA⋆	HotpotQA†	2wiki⋆	Musique⋆	Bamboogle⋆	Avg.
Qwen2.5-7b-Base/Instruct								
Direct Inference	0.134	0.408	0.140	0.183	0.250	0.031	0.120	0.181
CoT	0.048	0.185	0.054	0.092	0.111	0.022	0.232	0.106
IRCoT	0.224	0.478	0.301	0.133	0.149	0.072	0.224	0.239
Search-o1	0.151	0.443	0.131	0.187	0.176	0.058	0.296	0.206
RAG	0.349	0.585	0.392	0.299	0.235	0.058	0.208	0.304
SFT	0.318	0.354	0.121	0.217	0.259	0.066	0.112	0.207
R1-base	0.297	0.539	0.202	0.242	0.273	0.083	0.296	0.276
R1-instruct	0.270	0.537	0.199	0.237	0.292	0.072	0.293	0.271
Rejection Sampling	0.360	0.592	0.380	0.331	0.296	0.123	0.355	0.348
Search-R1-base	**0.480**	**0.638**	**0.457**	**0.433**	0.382	**0.196**	**0.432**	**0.431**
Search-R1-instruct	0.393	0.610	0.397	0.370	**0.414**	0.146	0.368	0.385
Qwen2.5-3b-Base/Instruct								
Direct Inference	0.106	0.288	0.108	0.149	0.244	0.020	0.024	0.134
CoT	0.023	0.032	0.005	0.021	0.021	0.002	0.000	0.015
IRCoT	0.111	0.312	0.200	0.164	0.171	0.067	0.240	0.181
Search-o1	0.238	0.472	0.262	0.221	0.218	0.054	**0.320**	0.255
RAG	0.348	0.544	0.387	0.255	0.226	0.047	0.080	0.270
SFT	0.249	0.292	0.104	0.186	0.248	0.044	0.112	0.176
R1-base	0.226	0.455	0.173	0.201	0.268	0.055	0.224	0.229
R1-instruct	0.210	0.449	0.171	0.208	0.275	0.060	0.192	0.224
Rejection Sampling	0.294	0.488	0.332	0.240	0.233	0.059	0.210	0.265
Search-R1-base	**0.406**	**0.587**	**0.435**	0.284	0.273	0.049	0.088	0.303
Search-R1-instruct	0.341	0.545	0.378	**0.324**	**0.319**	**0.103**	0.264	**0.325**

Table 2 from the paper

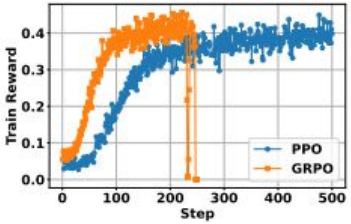
- SEARCH-R1 consistently beats strong baselines on average across the seven datasets.
- It outperforms standard retrieval baselines such as RAG, IRCoT, and Search-o1.
- It also improves over R1, which means the gain is not only from using RL, but from using RL to optimize search behavior during reasoning.

Interpretation

The overall pattern in Table 2 supports the paper's core claim: learned search policy is more effective than only attaching retrieval or prompting search at inference time.

PPO vs. GRPO

The paper compares two policy-gradient RL choices inside SEARCH-R1



(a) PPO vs. GRPO

Figure 2(a) from the paper

Table 3: The performance results of SEARCH-R1 with PPO and GRPO on seven datasets.

Method	NQ	TriviaQA	PopQA	HotpotQA	2wiki	Musique	Bamboogle	Avg.
Qwen2.5-7b-Base/Instruct								
SEARCH-R1-base (GRPO)	0.395	0.560	0.388	0.326	0.297	0.125	0.360	0.350
SEARCH-R1-instruct (GRPO)	0.429	0.623	0.427	0.386	0.346	0.162	0.400	0.396
SEARCH-R1-base (PPO)	**0.480**	**0.638**	**0.457**	**0.433**	0.382	**0.196**	**0.432**	**0.431**
SEARCH-R1-instruct (PPO)	0.393	0.610	0.397	0.370	**0.414**	0.146	0.368	0.385
Qwen2.5-3b-Base/Instruct								
SEARCH-R1-base (GRPO)	**0.421**	0.583	0.413	0.297	0.274	0.066	0.128	0.312
SEARCH-R1-instruct (GRPO)	0.397	0.565	0.391	**0.331**	0.310	**0.124**	0.232	**0.336**
SEARCH-R1-base (PPO)	0.406	**0.587**	**0.435**	0.284	0.273	0.049	0.088	0.303
SEARCH-R1-instruct (PPO)	0.341	0.545	0.378	0.324	**0.319**	0.103	**0.264**	0.325

Table 3 from the paper

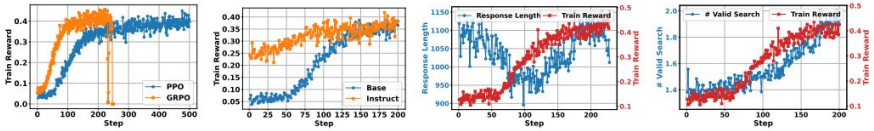
What is the difference?

PPO uses a critic or value model to estimate advantage, while GRPO uses a group of sampled outputs and relative rewards, so it does not need a learned value function.

What does the paper find?

GRPO converges faster, likely because it avoids PPO's critic warm-up. PPO is more stable over long training, while GRPO can show reward collapse later. Final performance is comparable and mixed across models, but the authors prefer PPO in this setting because of stability.

Additional analysis: search behavior and masking



(a) PPO vs. GRPO (b) Base vs. Instruct (c) Response length (d) # Valid search

Figure 2: (a) PPO vs. GRPO: GRPO generally converges faster but may exhibit instability after trained for a number of steps, whereas PPO provides more stable optimization but converges at a slower rate. (b) Base vs. Instruct LLM study: Instruction-tuned LLMs converge faster, but the final performance of both modles remains highly similar. (c) Response length study: The response length exhibits a decrease-increase-stabilize trend throughout training, aligning with the overall performance trajectory of the LLM. (d) # Valid search study: As the training proceeds, the LLM learns to call search more.

Figure 2 from the paper

Table 4: The performance of SEARCH-R1 with and without retrieved token loss masking. The LLM trained with retrieved token loss masking achieves consistently better performance. (LLM: Qwen2.5-7b-base; RL: PPO)

Method	NQ	TriviaQA	PopQA	HotpotQA	2wiki	Musique	Bamboogle	Avg.
SEARCH-R1 w. mask	**0.480**	**0.638**	**0.457**	**0.433**	**0.382**	**0.196**	**0.432**	**0.431**
SEARCH-R1 w.o. mask	0.388	0.567	0.391	0.325	0.321	0.108	0.304	0.343

Table 4 from the paper

- Figure 2 shows that valid search calls increase during training, which suggests the model is actually learning better search behavior rather than only generating longer answers.
- The base-versus-instruct comparison also shows that instruction-tuned models start stronger and converge faster, but RL narrows the gap over time.
- Table 4 shows that retrieved-token masking matters: performance drops clearly without masking, so it is a real part of why the method works.

Limitations and future directions

Limitation 1
The evaluation is still centered on QA-style benchmarks. It is not yet clear how well the framework transfers to broader agent tasks or less structured settings.

Limitation 2
The reward is simple and outcome-based. Future work could directly optimize faithfulness, search cost, latency, calibration, or uncertainty awareness.

Limitation 3
Real search is noisy and expensive. The paper does not fully answer when the model should stop searching or how to handle unreliable retrieval.

Future direction A
Extend the same idea from web search to broader tool use, such as APIs, databases, code execution, and multimodal retrieval.

Future direction B
Design richer rewards or adaptive stopping policies so the model knows when one more search is worth the extra cost.

Future direction C
Study how learned search policies generalize across models, domains, retrievers, and tool combinations.

Final takeaways

Takeaway 1
SEARCH-R1 shows that search behavior can be learned through RL, not just prompted at inference time.

Takeaway 2
The method is conceptually simple: multi-turn search rollouts, outcome-based reward, and retrieved-token masking.

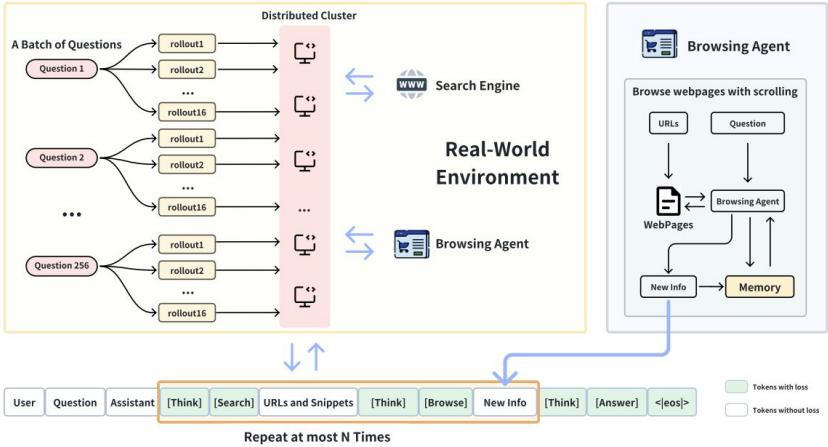
Takeaway 3
The main evidence comes from Table 2, while Figure 2, Table 3, and Table 4 explain how training behaves and why the method works.

Overall conclusion: SEARCH-R1 is not only a stronger QA system. It is also a useful example of how reinforcement learning can improve tool-use decisions inside reasoning, and that idea is likely to matter beyond search alone.

DeepResearcher:

Scaling Deep Research via Reinforcement Learning in Real-world Environments

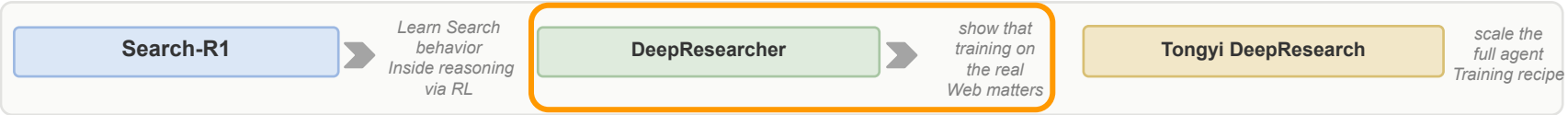
Yuxiang Zheng et al. | EMNLP 2025



Why Plain Retrieval is Not Enough for Deep Research?

- **Search-R1:** Models can interleave reasoning with search, but fail in real-world environments
- **Challenges:**
  - Traditional retrieval struggles with the dynamic, noisy open web
  - Requires full-page comprehension and multi-source synthesis
  - Systems must handle long-horizon planning under uncertainty

- **Motivation:** train agents in a real web environment so they learn not only when to search, but also how to browse, gather, and synthesize evidence like a research agent

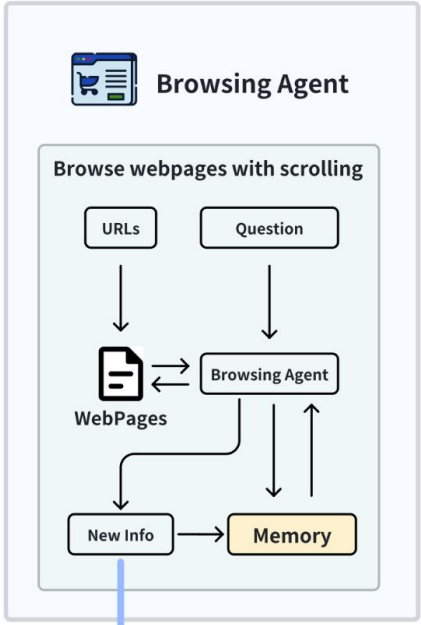


DeepResearcher: End-to-End RL in Real-World Environments

- The first framework for end-to-end training of LLM-based deep research agents through scaling RL in real-world environments with authentic web search interactions
- Moves RL training from a controlled corpus to the real web
  - Develops robust capabilities for **unpredictable, real-world info** retrieval and synthesis
  - In addition, **emergent behaviors** arises from end-to-end RL

Methodology: End-to-End Agent Pipeline

- Iteratively **reason → search → browse → synthesize**
  - The model is forced to **reason before acting** using <think> steps
  - It calls a **web search tool** with JSON-formatted queries
  - A **web browsing agent** reads webpage segments, stores relevant findings in short-term memory, and returns useful evidence
  - The model stops when it has enough information and outputs the final answer in <answer> tags

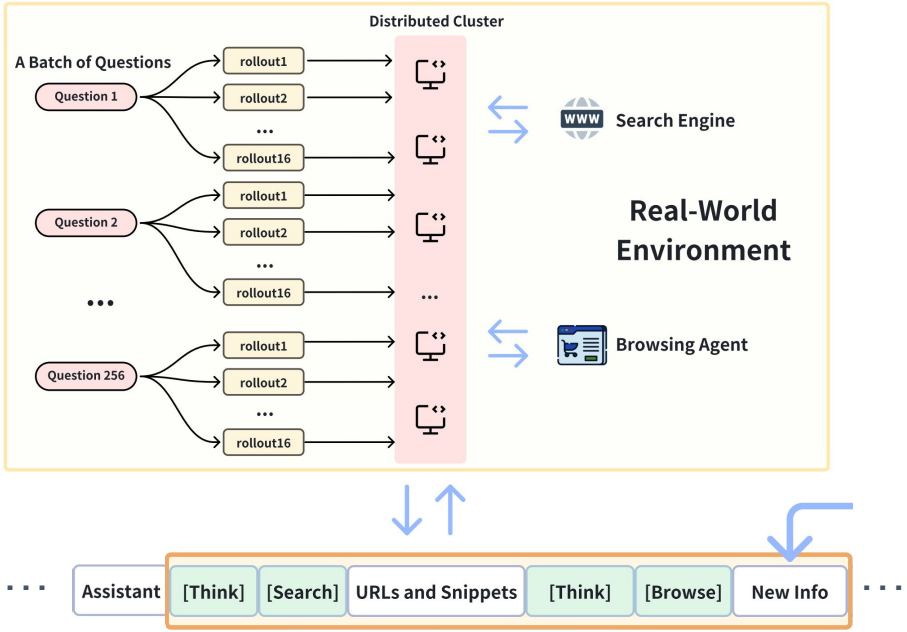


Repeat at most N Times

Methodology: Real-World Web Training Requires System Design

Real-web RL needs additional infrastructure

Real-web challenge	Why it matters	System design
Concurrency	RL rollouts trigger many simultaneous tool calls	50-node distributed CPU tool-serving cluster
Reliability	Search APIs and webpages fail, rate-limit, or block crawlers	Retry + caching for repeated queries
Page understanding	Full webpages are long and noisy	Dedicated browsing/reading agent with segmented extraction



Methodology: RL Training Framework

- Train DeepResearcher with **GRPO** over sampled research rollouts
- For each question, the policy generates multiple trajectories of reasoning, search, browsing, and answering

$$\mathcal{J}(\theta) = \mathbb{E}_{x\sim\mathcal{D},\{y_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|x)}$$
$$\frac{1}{G}\sum_{i=1}^{G}\left[\min\left(\frac{\pi_\theta(y_i|x)}{\pi_{\theta_{\text{old}}}(y_i|x)}A_i, \text{clip}\left(\frac{\pi_\theta(y_i|x)}{\pi_{\theta_{\text{old}}}(y_i|x)}, 1-\epsilon, 1+\epsilon\right)A_i\right) - \beta\mathbb{D}_{\text{KL}}\left(\pi_\theta||\pi_{\theta_{\text{ref}}}\right)\right],$$

y_i : rollout i, G : number of sampled rollouts, A_i : relative advantage, β : KL penalty weight

- **Observations from tools are masked** during optimization as they are not imitation target
- The model learns only from its own generated decisions and answers

Methodology: Reward

- Incorrect output format gets a penalty of **-1**
- Otherwise, reward is **word-level F1** against the reference answer

$$\text{reward} = \begin{cases} -1 & \text{if format is incorrect} \\ \text{F1 score} & \text{if format is correct} \end{cases}$$

Methodology: Curating Search-Dependent Training Data

“How do we create training data that forces the model to learn search, rather than exploit memorization?”

- Build training data from open-domain QA benchmarks (NQ + TriviaQA + HotpotQA + 2Wiki)
- Remove low-quality questions and detect contamination
  - Eliminate:
    1) Time-sensitive questions (e.g., “Who is the current CEO of Apple?”)
    2) Highly subjective queries (e.g., “What is the best smartphone?”)
    3) Potentially policy-violating content
- Keep examples that genuinely require web search and multi-hop evidence synthesis

Experimental Setup

- Backbone: Qwen2.5-7B-Instruct
- In-domain (training-domain) benchmarks: NQ, TriviaQA, HotpotQA, 2WikiMultiHopQA
- Out-of-domain benchmarks: MuSiQue, Bamboogle, PopQA
- Baselines: CoT, CoT+RAG, Search-o1, Search-o1 + Web Search, Search-R1, R1-Searcher
- Metrics: F1 and MBE (model-based evaluation)
- MBE: GPT-4o-mini is prompted with the question, ground-truth answer, and model answer, then labels the answer as correct or incorrect; the MBE score is the accuracy of those judgments

Main Results Overview

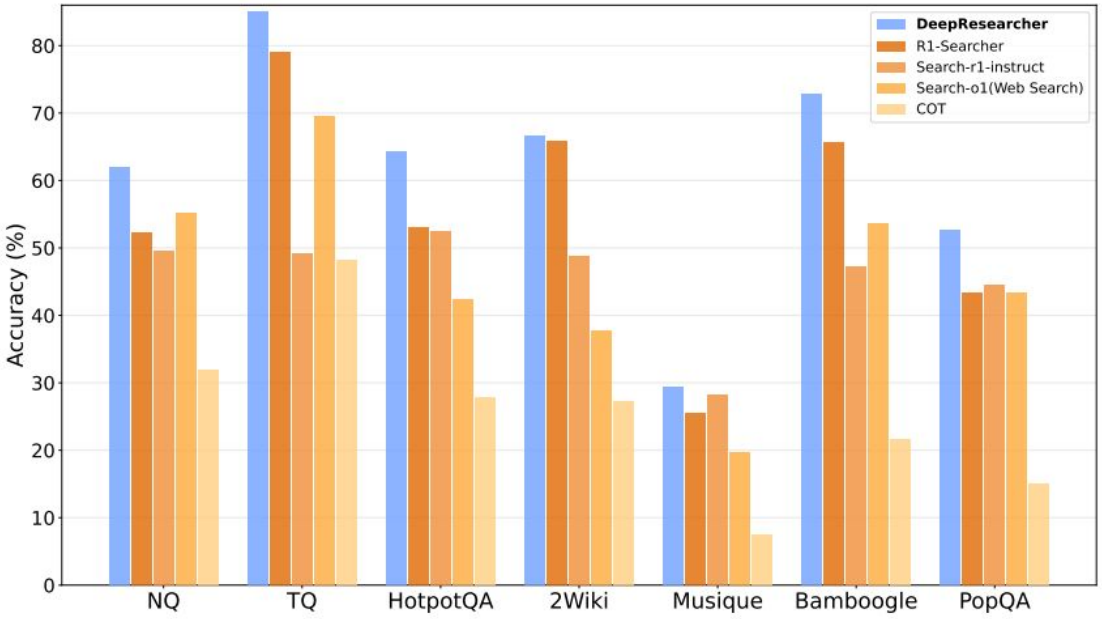


Figure 1: DeepResearcher performs the best on all 7 datasets measured by reliable model-based evaluation.

In-Domain Results

- DeepResearcher leads on all 4 in-domain datasets under MBE
- F1 is competitive overall, with especially strong gains on TriviaQA and 2Wiki

Method	Inference Environment	NQ		TQ		HotpotQA		2Wiki	
		F1	MBE	F1	MBE	F1	MBE	F1	MBE
Prompt Based									
CoT	Local RAG	19.8	32.0	45.6	48.2	24.4	27.9	26.4	27.3
CoT + RAG	Local RAG	42.0	59.6	68.9	75.8	37.1	43.8	24.4	24.8
Search-o1*	Local RAG	34.5	57.4	52.6	61.1	31.6	40.8	28.6	32.8
Search-o1	Web Search	32.4	55.1	58.9	69.5	33.0	42.4	30.9	37.7
Training Based									
Search-r1-base	Local RAG	**45.4**	60.0	71.9	76.2	**55.9**	63.0	44.6	47.9
Search-r1-instruct	Local RAG	33.1	49.6	44.7	49.2	45.7	52.5	43.4	48.8
R1-Searcher	Web Search	35.4	52.3	73.1	79.1	44.8	53.1	59.4	65.8
DeepResearcher	Web Search	39.6	**61.9**	**78.4**	**85.0**	52.8	**64.3**	**59.7**	**66.6**

Out-of-Domain Results / Generalization

- DeepResearcher leads in both F1 and MBE on all datasets
  - This shows training on real web environments generalizes better than controlled settings

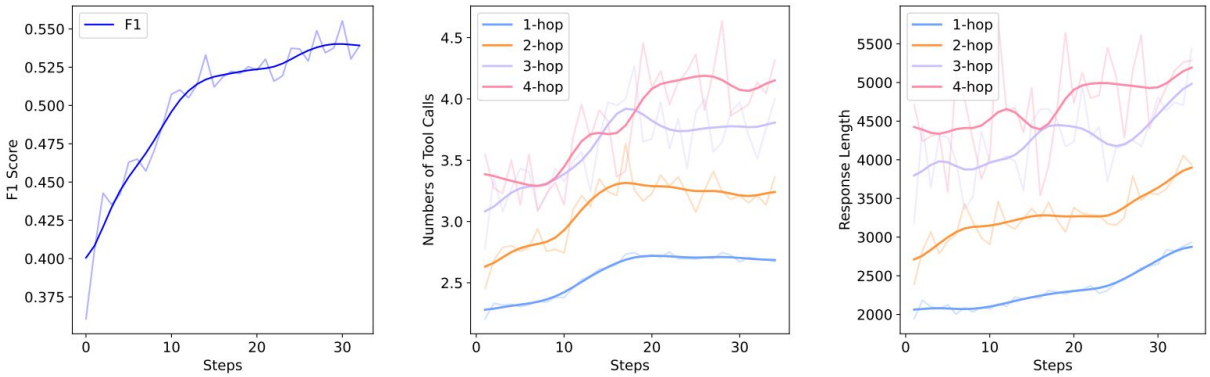
Method	Inference Environment	Musique		Bamboogle		PopQA	
		F1	MBE	F1	MBE	F1	MBE
Prompt Based							
CoT	Local RAG	8.5	7.4	22.1	21.6	17.0	15.0
CoT + RAG	Local RAG	10.0	10.0	25.4	27.2	46.9	48.8
Search-o1*	Local RAG	16.8	21.3	35.8	38.4	36.9	42.4
Search-o1	Web Search	14.7	19.7	46.6	53.6	38.3	43.4
Training Based							
Search-r1-base	Local RAG	26.7	27.5	56.5	57.6	43.2	47.0
Search-r1-instruct	Local RAG	26.5	28.3	45.0	47.2	43.0	44.5
R1-Searcher	Web Search	22.8	25.6	64.8	65.6	42.7	43.4
DeepResearcher	Web Search	**27.1**	**29.3**	**71.0**	**72.8**	**48.5**	**52.7**

Analysis: Training Dynamics

a) RL steadily improves performance

b) Harder questions trigger more tool use

c) Response length keeps growing during training



(a) (b) (c)

Case Study: Emergent Research Behaviors from End-to-End RL

RL seems to improve not only accuracy, but also **research behavior**

- Following emergent behaviors were observed:
  1. **Planning** when addressing multi-hop questions
  2. **Cross-validation** before finalizing its answers
  3. **Reflection** when observations deviate from expectations
  4. **Honesty** by acknowledging its limitations

- **These behaviors are not explicitly supervised**

Case Study: Emergent Research Behaviors from End-to-End RL

Pre-RL models do NOT do this structured planning

- Example: Planning when addressing multi-hop questions
  - **Question:** What is the name of the famous bridge in the birthplace of Gloria in D Major's composer?
  - **Ground Truth:** Rialto Bridge

Step 1: Identify the composer of Gloria in D Major.
Step 2: Find the place of birth of that composer.
Step 3: Look for a famous bridge in that city.

Make a plan first

...(more results are omitted to save space)
</tool_response>
<think>
From the search results, **I can see that Antonio Vivaldi is the composer of Gloria in D Major** and **he was born in Venice**. Now I need to find the famous bridge in Venice.
</think>
<search>
famous bridge in Venice

Step 1

Step 2

</tool_response>
<think>
The Rialto Bridge is the oldest and most famous bridge in Venice. I can now answer the question.
</think>
<answer>
Rialto Bridge
</answer>

Step 3

Limitations

- Evaluated only on a 7B backbone
- Reward is tailored to short-answer QA, not fully open-ended deep research
- Training data is built mainly from open-domain QA benchmarks, so it may not fully reflect the complexity of real deep-research tasks

Conclusion

- DeepResearcher trains research agents end-to-end in real-world web environments
- Real-web RL improves both benchmark performance and research behavior
- Authentic web interaction is a core training requirement, not just an implementation detail

Tongyi DeepResearch Technical Report

An instance for deep research implementation

Tongyi lab | Alibaba Group

Key Innovations

End-to-end agentic training paradigm

Full cycle agentic training unifying mid-training and post-training, enabling the model to gradually develop from basic interaction skills to advanced autonomous research behaviors.

Fully automated, highly scalable data synthesis pipeline

The model is trained without direct human annotation which creates highly scalable synthetic data and rapid validation, constructing superhuman-level datasets

Stage-specific, customized environments

Allowing the model to engage in rich, specialized interactions that are tightly aligned with its developmental stage

Learning through Environmental Interactions

- Prior World Environment
  Providing task elements, tools, and state definitions
  Lacking real-world feedback

- Simulated Environment
  Controlled, reproducible replicas of real-world interactions
  Data coverage is inherently limited, exhibiting a notable sim-to-real gap

- Real-world Environment
  Most authentic data distribution and feedback signals
  Expensive interactions, significant non-stationarity, and exploration risks

For Mid Training

For Post Training

Search-R1 (COLM 2025)

Formulation

Time components in the thinking progress

1. Thought

   Context analyzing, response steps planning and self reflection

2. Action

   Environmental interaction

3. Observation

   Feedbacks from the action

Rollouts

1. ReAct

   Vanilla ReAct $\mathcal{H}_T = (\tau_0, a_0, o_0, \ldots, \tau_i, a_i, o_i, \ldots, \tau_T, a_T)$ could cause context overflow

2. Context Management

   By applying a Markovian state reconstruction it changes to $S_t, \tau_{t+1}, a_{t+1} \sim \pi(\cdot | S_{t-1}, a_t, o_t)$.

   The model sees an evolving process instead of whole history

   More applicable for long context scenario

Training Process

Pre-training: Qwen3-30B-A3B-Base as a pre-trained model

Mid-Training

Stage 1: Initiates the optimization process with a 32K context length.
Stage 2: Expands the context window to 128K by introducing a substantial corpus of long-sequence agentic behavior data.

Post-Training

Stage 1: Supervised Fine-Tuning
Stage 2: Reinforce learning

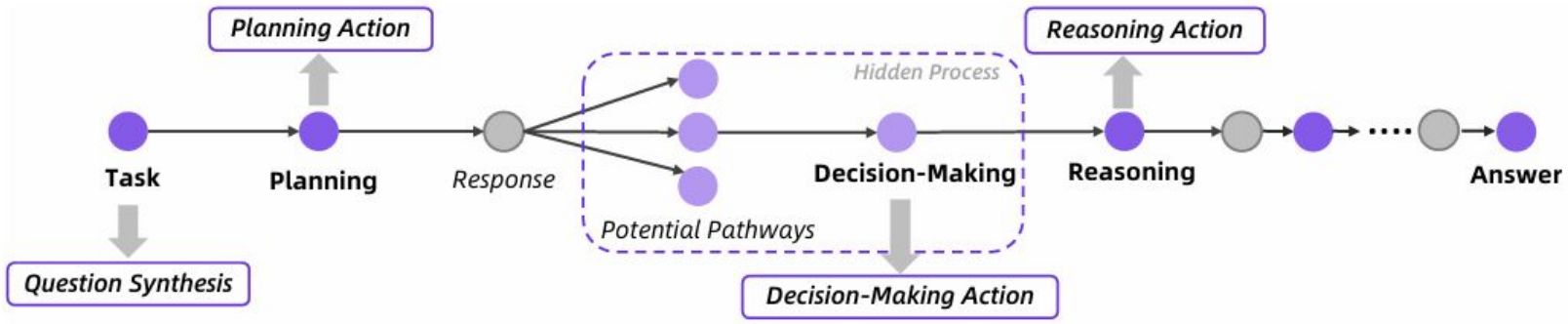
Mid Training

Two-Stage Agentic Continual Pre-training

Initiate with a 32K context length in the first stage, 128K in second stage

Planning, Reasoning and Decision-Making Actions are explicitly demonstrated

Function-calling Data Synthesis via Environment Scaling

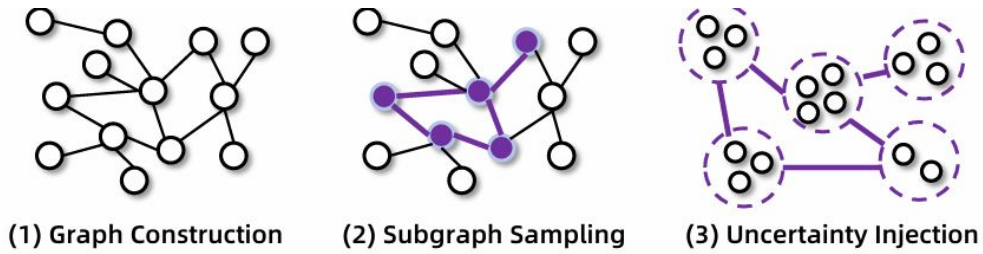


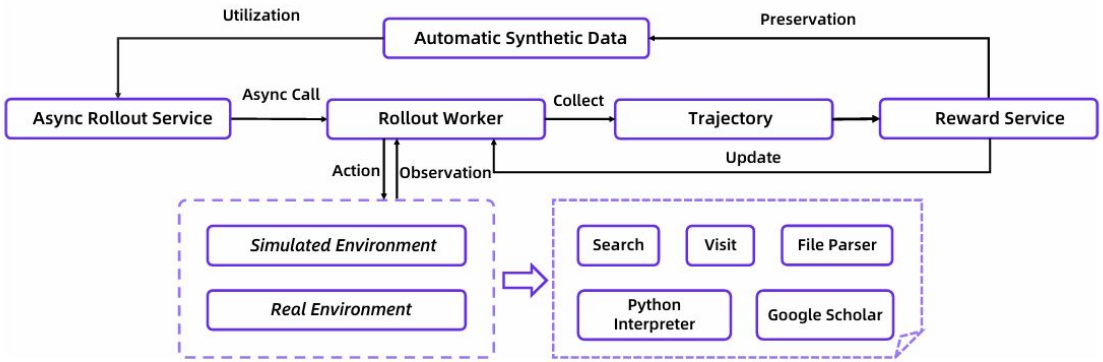
Post Training

Data synthesis:
Select knowledge subgraphs in an knowledge graph
Introduce uncertainty to generate PhD-level questions automatically

Supervised Fine-Tuning:
Both ReAct and Context Management are used
Context Management show strength in longer context

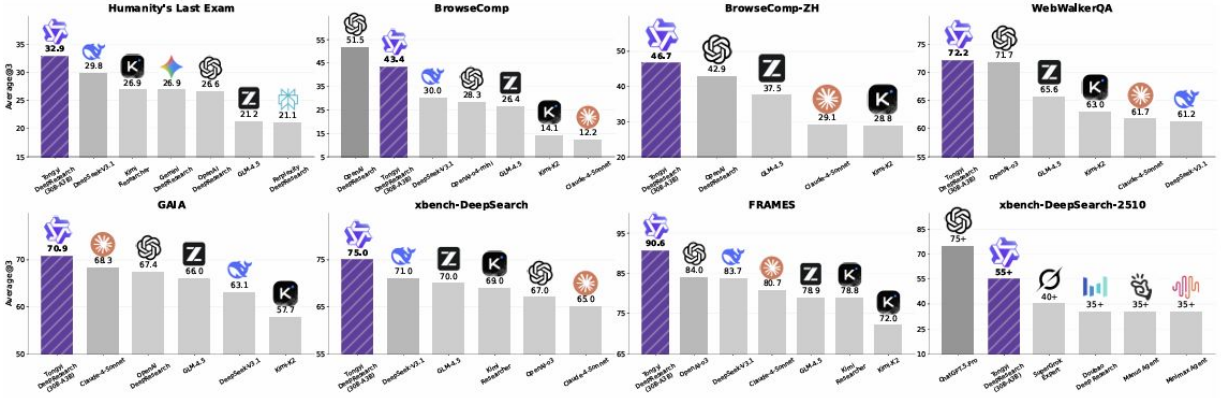
Reinforced Learning:
Applies a tailored adaptation of the GRPO algorithm, using a pure 0 or 1 reward signal based solely on final answer correctness.





Performance: Benchmark of the model

30.5B parameters
Only 3.3B activated per token
Excellent performance in
Benchmark tests



Benchmarks	Humanity's Last Exam	Browse Comp	Browse Comp-ZH	GAIA	xbench DeepSearch	WebWalker QA	FRAMES
LLM-based ReAct Agent							
GLM 4.5	21.2	26.4	37.5	66.0	70.0	65.6	78.9
Kimi K2	18.1	14.1	28.8	57.7	50.0	63.0	72.0
DeepSeek-V3.1	29.8	30.0	49.2	63.1	71.0	61.2	83.7
Claude-4-Sonnet	20.3	12.2	29.1	68.3	65.0	61.7	80.7
OpenAI o3	24.9	49.7	58.1	–	67.0	71.7	84.0
OpenAI o4-mini	17.7	28.3	–	60.0	–	–	–
DeepResearch Agent							
OpenAI DeepResearch	26.6	51.5	42.9	67.4	–	–	–
Gemini DeepResearch	26.9	–	–	–	–	–	–
Kimi Researcher	26.9	–	–	–	69.0	–	78.8
Tongyi DeepResearch (30B-A3B)	**32.9**	**43.4**	**46.7**	**70.9**	**75.0**	**72.2**	**90.6**

Performance: Heavy Mode

Parallel Research Phase:

n parallel agents are deployed

Each agent u is tasked to independently solve question q

Results are compressed in S_T^u

$$(S_T^u, \text{answer}_u) = \text{Agent}_u(q), \quad u \in [1, n]$$

Integrative Synthesis Phase

A synthesis model consolidates all parallel findings to produce the final answer

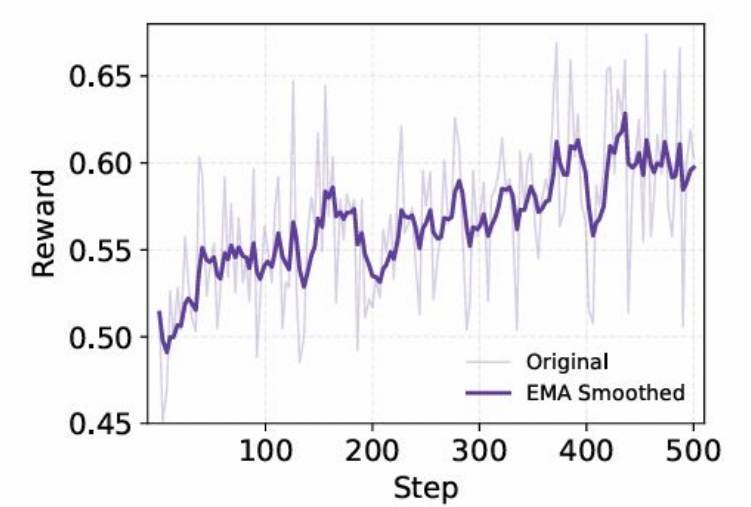
$$\text{answer}_{\text{final}} = \text{Synthesis}\left(\{(S_T^u, \text{answer}_u)\}_{u=1}^n\right)$$

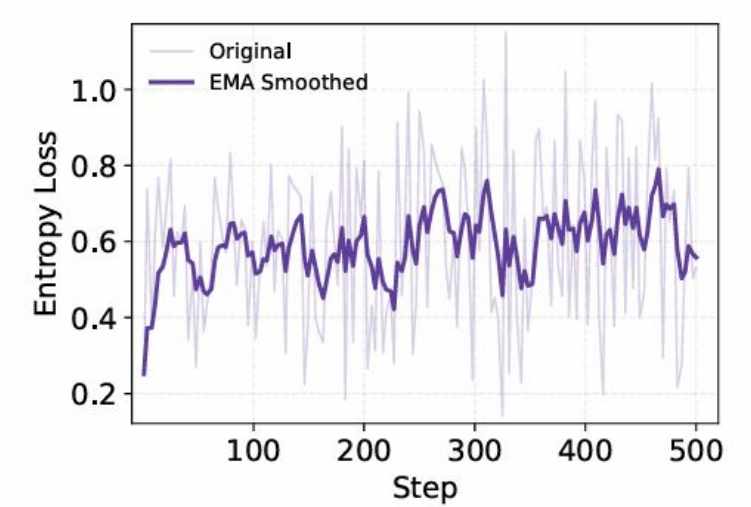
Assessing n diverse solution strategies within a manageable context window without exceeding context

Performance Analysis- Rewards and Entropy

Performance boast as more training conducted

Stable policy entropy, consistent increase after initial boom

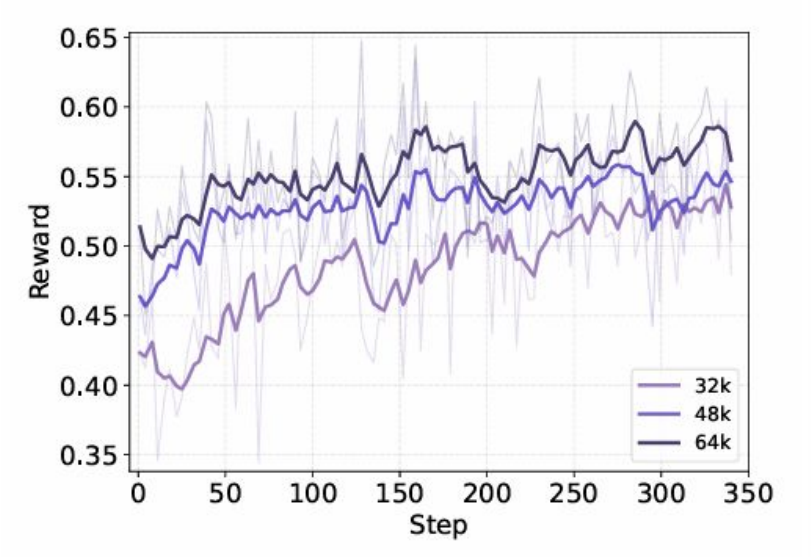


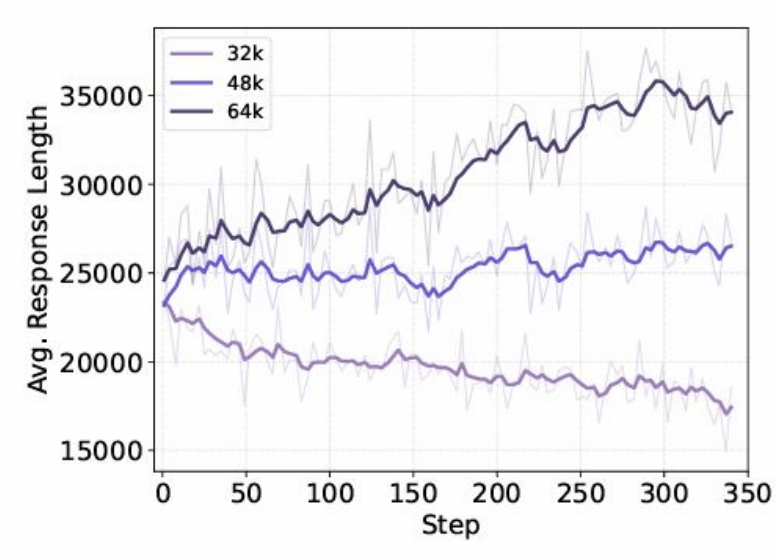


Performance Analysis- Reward and Response length

Reward is roughly the same trend across contexts
64K context has highest reward
32K and 48K context are limited by length

64K context increases with steps
48K context remains roughly the same
32K context decreases with steps

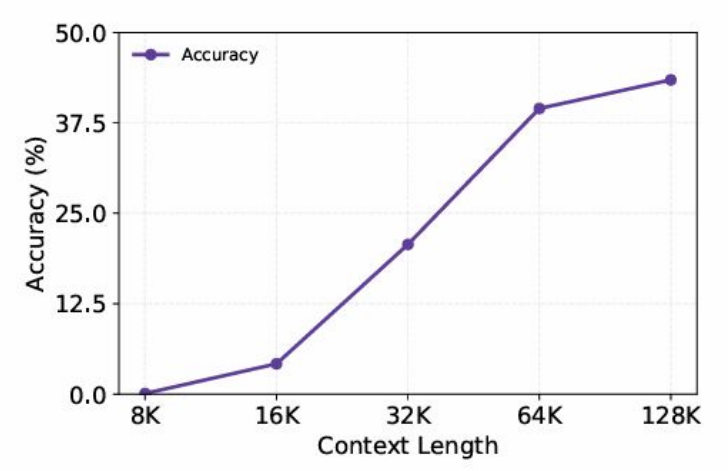


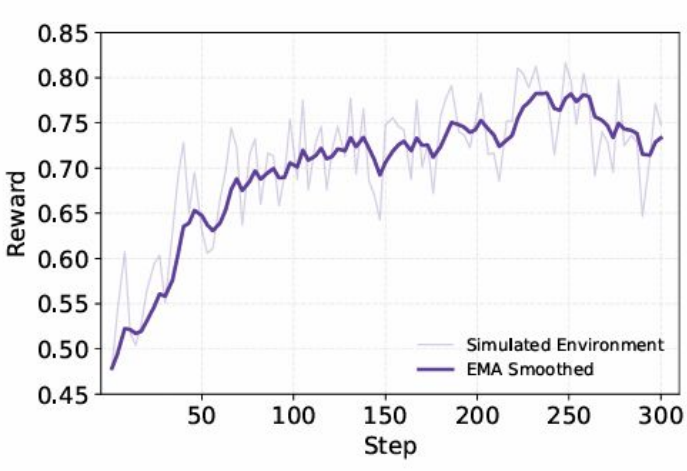


Performance Analysis- Environmental Interaction Results

Performance on the BrowseComp dataset improves consistently with more context length and interactions

GRPO algorithm outcome matches the simulation





Limitation of the model

Limitation 1
The model has only 128K context length which could be insufficient for handling more complex tasks

Limitation 2
The scale of the model is currently quite small, a larger model is WIP

Limitation 3
The report generation fidelity and user preferences could be further optimized

Limitation 4
The efficiency of the reinforcement learning framework could be further improved by partial rollouts

Limitation 5
The prompt and toolsets of the deep research is fixed, further training with different toolsets and prompts can provide more versatility and robustness.

Future works

Future direction A

Extend the scenario from domain-specific agents to general-purpose agents

Future direction B

Although the smaller model is behaving excellently as an agent, a larger model is currently in progress.

Possible Future direction

Extend the context window by further context management

Conclusion

One theme, three steps

The papers can be read as a progression in what it takes to build a deep research agent.

1. Search-R1

Core idea
Train search behavior with reinforcement learning inside the reasoning trajectory.

Why it matters
Search stops being a hand-written tool call and becomes a learned decision.

2. DeepResearcher

Core idea
Move training from a controlled retrieval corpus to the open web.

Why it matters
The real web adds noise, structure variation, and source conflicts that deep research must handle.

3. Tongyi DeepResearch

Core idea
Scale a complete agent training stack with data synthesis, staged environments, and agentic training.

Why it matters
Strong deep research performance needs more than one RL trick.

Overall takeaway The shift is from attaching retrieval to an LLM, toward training an agent that can search, browse, verify, and synthesize information over long horizons.

Thanks For Listening!
Questions?